

校庆纪念品认购私域微信小程序 — 详细设计文档

版本：1.0 | 日期：2026-03-09

目录

- 1. 数据库详细表设计
- 2. 核心业务流程详细设计
- 3. 关键算法设计
- 4. 需求对照表

1. 数据库详细表设计

1.1 users 用户表

描述：存储微信小程序用户的基本信息，包括境内外校友。

```
CREATE TABLE `users` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '用户ID',  
  `openid` VARCHAR(64) NOT NULL COMMENT '微信OpenID',  
  `union_id` VARCHAR(64) DEFAULT NULL COMMENT '微信UnionID',  
  `nickname` VARCHAR(100) DEFAULT NULL COMMENT '昵称',  
  `avatar_url` VARCHAR(500) DEFAULT NULL COMMENT '头像URL',  
  `phone` VARCHAR(20) DEFAULT NULL COMMENT '手机号',  
  `country_code` VARCHAR(10) DEFAULT '+86' COMMENT '国家区号',  
  `is_overseas` TINYINT(1) DEFAULT 0 COMMENT '是否海外用户: 0否 1是',  
  `is_verified` TINYINT(1) DEFAULT 0 COMMENT '是否已通过校友认证: 0否 1是',  
  `status` TINYINT DEFAULT 1 COMMENT '账号状态: 1正常 0禁用',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',  
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `idx_openid` (`openid`),  
  KEY `idx_union_id` (`union_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	用户ID
openid	VARCHAR(64)	UNIQUE NOT NULL	微信OpenID
union_id	VARCHAR(64)	NULL	微信UnionID
nickname	VARCHAR(100)	NULL	昵称
avatar_url	VARCHAR(500)	NULL	头像URL
phone	VARCHAR(20)	NULL	手机号
country_code	VARCHAR(10)	DEFAULT '+86'	国家区号

字段名	类型	约束	说明
is_overseas	TINYINT(1)	DEFAULT 0	是否海外用户
is_verified	TINYINT(1)	DEFAULT 0	是否校友认证通过
status	TINYINT	DEFAULT 1	账号状态
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

索引： idx_openid （唯一）、 idx_union_id

1.2 alumni_verifications 校友认证表

描述：记录用户提交的校友身份认证申请及审核结果。

```
CREATE TABLE `alumni_verifications` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '认证记录ID',  
  `user_id` BIGINT NOT NULL COMMENT '用户ID',  
  `real_name` VARCHAR(100) NOT NULL COMMENT '真实姓名',  
  `student_id` VARCHAR(50) DEFAULT NULL COMMENT '学号',  
  `graduation_year` YEAR DEFAULT NULL COMMENT '毕业年份',  
  `major` VARCHAR(100) DEFAULT NULL COMMENT '专业',  
  `proof_image_url` VARCHAR(500) DEFAULT NULL COMMENT '证明材料图片URL',  
  `status` ENUM('pending','approved','rejected') NOT NULL DEFAULT 'pending' COMMENT '审核状态',  
  `reviewer_id` BIGINT DEFAULT NULL COMMENT '审核管理员ID',  
  `review_remark` VARCHAR(500) DEFAULT NULL COMMENT '审核备注',  
  `reviewed_at` DATETIME DEFAULT NULL COMMENT '审核时间',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '申请时间',  
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',  
  PRIMARY KEY (`id`),  
  KEY `idx_user_id` (`user_id`),  
  KEY `idx_status` (`status`),  
  KEY `idx_student_id` (`student_id`),  
  CONSTRAINT `fk_av_user_id` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`),  
  CONSTRAINT `fk_av_reviewer_id` FOREIGN KEY (`reviewer_id`) REFERENCES `admins` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='校友认证表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	认证记录ID
user_id	BIGINT	FK users.id NOT NULL	用户ID
real_name	VARCHAR(100)	NOT NULL	真实姓名
student_id	VARCHAR(50)	NULL	学号
graduation_year	YEAR	NULL	毕业年份
major	VARCHAR(100)	NULL	专业
proof_image_url	VARCHAR(500)	NULL	证明材料图片URL
status	ENUM	DEFAULT 'pending'	审核状态： pending/approved/rejected

字段名	类型	约束	说明
reviewer_id	BIGINT	FK admins.id NULL	审核管理员ID
review_remark	VARCHAR(500)	NULL	审核备注
reviewed_at	DATETIME	NULL	审核时间
created_at	DATETIME	NOT NULL	申请时间
updated_at	DATETIME	NOT NULL	更新时间

索引： `idx_user_id` 、 `idx_status` 、 `idx_student_id`

外键： `user_id` → [users.id](#), `reviewer_id` → [admins.id](#)

1.3 products 商品表

描述：存储校庆纪念品商品主信息。

```
CREATE TABLE `products` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '商品ID',  
  `name` VARCHAR(200) NOT NULL COMMENT '商品名称（中文）',  
  `name_en` VARCHAR(200) DEFAULT NULL COMMENT '商品名称（英文）',  
  `description` TEXT DEFAULT NULL COMMENT '商品简介',  
  `detail_html` LONGTEXT DEFAULT NULL COMMENT '图文详情HTML',  
  `category_id` BIGINT DEFAULT NULL COMMENT '分类ID',  
  `cover_image` VARCHAR(500) DEFAULT NULL COMMENT '封面图URL',  
  `price` DECIMAL(10,2) NOT NULL COMMENT '基准价格（CNY）',  
  `stock` INT DEFAULT 0 COMMENT '总库存（冗余字段）',  
  `sales_count` INT DEFAULT 0 COMMENT '累计销量',  
  `is_customizable` TINYINT(1) DEFAULT 0 COMMENT '是否支持刻字定制',  
  `status` ENUM('draft','published','unpublished') NOT NULL DEFAULT 'draft' COMMENT '状态',  
  `sort_order` INT DEFAULT 0 COMMENT '排序权重，越大越靠前',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  KEY `idx_category_id` (`category_id`),  
  KEY `idx_status` (`status`),  
  KEY `idx_sort_order` (`sort_order`),  
  CONSTRAINT `fk_products_category` FOREIGN KEY (`category_id`) REFERENCES `product_categories` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='商品表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	商品ID
name	VARCHAR(200)	NOT NULL	商品名称（中文）
name_en	VARCHAR(200)	NULL	商品名称（英文）
description	TEXT	NULL	商品简介
detail_html	LONGTEXT	NULL	图文详情HTML
category_id	BIGINT	FK <code>product_categories.id</code>	分类ID
cover_image	VARCHAR(500)	NULL	封面图URL

字段名	类型	约束	说明
price	DECIMAL(10,2)	NOT NULL	基准价格（CNY）
stock	INT	DEFAULT 0	总库存冗余字段
sales_count	INT	DEFAULT 0	累计销量
is_customizable	TINYINT(1)	DEFAULT 0	是否支持刻字定制
status	ENUM	DEFAULT 'draft'	状态：draft/published/unpublished
sort_order	INT	DEFAULT 0	排序权重
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

索引： idx_category_id 、 idx_status 、 idx_sort_order

1.4 product_skus 商品SKU表

描述：存储商品各规格组合（SKU）的独立库存与价格。

```
CREATE TABLE `product_skus` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT 'SKU ID',  
  `product_id` BIGINT NOT NULL COMMENT '商品ID',  
  `sku_code` VARCHAR(100) NOT NULL COMMENT 'SKU编码',  
  `spec_values` JSON DEFAULT NULL COMMENT '规格值，如{"color":"红色","size":"大"}',  
  `price` DECIMAL(10,2) NOT NULL COMMENT 'SKU价格（CNY）',  
  `stock` INT DEFAULT 0 COMMENT '库存数量',  
  `image_url` VARCHAR(500) DEFAULT NULL COMMENT 'SKU图片URL',  
  `status` TINYINT DEFAULT 1 COMMENT '状态：1启用 0禁用',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `idx_sku_code` (`sku_code`),  
  KEY `idx_product_id` (`product_id`),  
  CONSTRAINT `fk_skus_product` FOREIGN KEY (`product_id`) REFERENCES `products` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='商品SKU表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	SKU ID
product_id	BIGINT	FK products.id NOT NULL	商品ID
sku_code	VARCHAR(100)	UNIQUE NOT NULL	SKU编码
spec_values	JSON	NULL	规格值JSON
price	DECIMAL(10,2)	NOT NULL	SKU价格
stock	INT	DEFAULT 0	库存数量
image_url	VARCHAR(500)	NULL	SKU图片URL
status	TINYINT	DEFAULT 1	启用状态

字段名	类型	约束	说明
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

索引： idx_sku_code （唯一）、 idx_product_id

1.5 product_categories 商品分类表

描述： 商品分类，支持两级层级结构。

```
CREATE TABLE `product_categories` (  
  `id`          BIGINT      NOT NULL AUTO_INCREMENT COMMENT '分类ID',  
  `name`        VARCHAR(100) NOT NULL COMMENT '分类名称',  
  `parent_id`   BIGINT      DEFAULT NULL COMMENT '父分类ID, NULL为一级分类',  
  `icon_url`    VARCHAR(500) DEFAULT NULL COMMENT '分类图标URL',  
  `sort_order`  INT          DEFAULT 0 COMMENT '排序权重',  
  `status`      TINYINT     DEFAULT 1 COMMENT '状态: 1启用 0禁用',  
  `created_at`  DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at`  DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  KEY `idx_parent_id` (`parent_id`),  
  CONSTRAINT `fk_category_parent` FOREIGN KEY (`parent_id`) REFERENCES `product_categories` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='商品分类表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	分类ID
name	VARCHAR(100)	NOT NULL	分类名称
parent_id	BIGINT	NULL 自引用	父分类ID
icon_url	VARCHAR(500)	NULL	分类图标URL
sort_order	INT	DEFAULT 0	排序权重
status	TINYINT	DEFAULT 1	启用状态
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

1.6 product_images 商品图片表

描述： 存储商品轮播图及详情图。

```
CREATE TABLE `product_images` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '图片ID',  
  `product_id` BIGINT NOT NULL COMMENT '商品ID',  
  `image_url` VARCHAR(500) NOT NULL COMMENT '图片URL',  
  `sort_order` INT DEFAULT 0 COMMENT '排序权重',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  KEY `idx_product_id` (`product_id`),  
  CONSTRAINT `fk_images_product` FOREIGN KEY (`product_id`) REFERENCES `products` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='商品图片表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	图片ID
product_id	BIGINT	FK products.id NOT NULL	商品ID
image_url	VARCHAR(500)	NOT NULL	图片URL
sort_order	INT	DEFAULT 0	排序权重
created_at	DATETIME	NOT NULL	创建时间

1.7 addresses 收货地址表

描述：用户收货地址，支持境内外地址。

```
CREATE TABLE `addresses` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '地址ID',  
  `user_id` BIGINT NOT NULL COMMENT '用户ID',  
  `receiver_name` VARCHAR(100) NOT NULL COMMENT '收件人姓名',  
  `phone` VARCHAR(20) NOT NULL COMMENT '联系电话',  
  `country` VARCHAR(100) DEFAULT '中国' COMMENT '国家',  
  `province` VARCHAR(100) DEFAULT NULL COMMENT '省份/州',  
  `city` VARCHAR(100) DEFAULT NULL COMMENT '城市',  
  `district` VARCHAR(100) DEFAULT NULL COMMENT '区/县',  
  `detail` VARCHAR(500) NOT NULL COMMENT '详细地址',  
  `postal_code` VARCHAR(20) DEFAULT NULL COMMENT '邮政编码',  
  `is_default` TINYINT(1) DEFAULT 0 COMMENT '是否默认地址',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  KEY `idx_user_id` (`user_id`),  
  CONSTRAINT `fk_addr_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='收货地址表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	地址ID
user_id	BIGINT	FK users.id NOT NULL	用户ID
receiver_name	VARCHAR(100)	NOT NULL	收件人姓名
phone	VARCHAR(20)	NOT NULL	联系电话

字段名	类型	约束	说明
country	VARCHAR(100)	DEFAULT '中国'	国家
province	VARCHAR(100)	NULL	省份/州
city	VARCHAR(100)	NULL	城市
district	VARCHAR(100)	NULL	区/县
detail	VARCHAR(500)	NOT NULL	详细地址
postal_code	VARCHAR(20)	NULL	邮政编码
is_default	TINYINT(1)	DEFAULT 0	是否默认地址
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

索引： idx_user_id

1.8 carts 购物车表

描述：每位用户对应唯一一个购物车。

```
CREATE TABLE `carts` (  
  `id`          BIGINT  NOT NULL AUTO_INCREMENT COMMENT '购物车ID',  
  `user_id`     BIGINT  NOT NULL COMMENT '用户ID',  
  `created_at`  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at`  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `idx_user_id` (`user_id`),  
  CONSTRAINT `fk_cart_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='购物车表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	购物车ID
user_id	BIGINT	FK users.id UNIQUE	用户ID（一人一车）
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

1.9 cart_items 购物车明细表

描述：购物车中每个SKU的加购记录，可含刻字定制信息。

```
CREATE TABLE `cart_items` (
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '明细ID',
  `cart_id` BIGINT NOT NULL COMMENT '购物车ID',
  `product_id` BIGINT NOT NULL COMMENT '商品ID',
  `sku_id` BIGINT NOT NULL COMMENT 'SKU ID',
  `quantity` INT NOT NULL DEFAULT 1 COMMENT '数量',
  `customization_text` VARCHAR(200) DEFAULT NULL COMMENT '刻字内容',
  `selected_font` VARCHAR(50) DEFAULT NULL COMMENT '选择的字体',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uniq_cart_sku` (`cart_id`, `sku_id`),
  KEY `idx_cart_id` (`cart_id`),
  CONSTRAINT `fk_ci_cart` FOREIGN KEY (`cart_id`) REFERENCES `carts` (`id`),
  CONSTRAINT `fk_ci_product` FOREIGN KEY (`product_id`) REFERENCES `products` (`id`),
  CONSTRAINT `fk_ci_sku` FOREIGN KEY (`sku_id`) REFERENCES `product_skus` (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='购物车明细表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	明细ID
cart_id	BIGINT	FK carts.id NOT NULL	购物车ID
product_id	BIGINT	FK products.id NOT NULL	商品ID
sku_id	BIGINT	FK product_skus.id NOT NULL	SKU ID
quantity	INT	DEFAULT 1	数量
customization_text	VARCHAR(200)	NULL	刻字内容
selected_font	VARCHAR(50)	NULL	选择的字体
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

索引： idx_cart_id 、 UNIQUE(cart_id , sku_id)

1.10 orders 订单主表

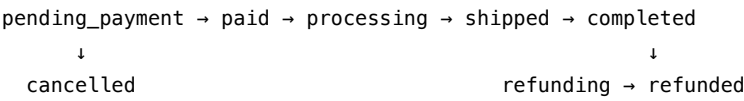
描述： 订单核心信息，记录下单金额、支付方式、状态流转及地址快照。


```
CREATE TABLE `orders` (
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '订单ID',
  `order_no` VARCHAR(32) NOT NULL COMMENT '订单编号',
  `user_id` BIGINT NOT NULL COMMENT '用户ID',
  `total_amount` DECIMAL(10,2) NOT NULL COMMENT '订单总金额 (CNY) ',
  `currency` VARCHAR(10) DEFAULT 'CNY' COMMENT '支付货币',
  `exchange_rate` DECIMAL(10,4) DEFAULT NULL COMMENT '下单时汇率 (海外订单) ',
  `address_id` BIGINT DEFAULT NULL COMMENT '收货地址ID',
  `address_snapshot` JSON DEFAULT NULL COMMENT '下单时地址快照',
  `status` ENUM('pending_payment','paid','processing','shipped','completed','cancelled','refunding','refunded') DEFAULT 'pending_payment' COMMENT '订单状态',
  `payment_method` ENUM('wechat','paypal') DEFAULT NULL COMMENT '支付方式',
  `payment_id` BIGINT DEFAULT NULL COMMENT '支付记录ID',
  `remark` VARCHAR(500) DEFAULT NULL COMMENT '买家备注',
  `cancel_reason` VARCHAR(200) DEFAULT NULL COMMENT '取消原因',
  `expires_at` DATETIME DEFAULT NULL COMMENT '支付超时时间',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `idx_order_no` (`order_no`),
  KEY `idx_user_id` (`user_id`),
  KEY `idx_status` (`status`),
  KEY `idx_expires_at` (`expires_at`),
  CONSTRAINT `fk_orders_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`),
  CONSTRAINT `fk_orders_addr` FOREIGN KEY (`address_id`) REFERENCES `addresses` (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='订单主表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	订单ID
order_no	VARCHAR(32)	UNIQUE NOT NULL	订单编号
user_id	BIGINT	FK users.id NOT NULL	用户ID
total_amount	DECIMAL(10,2)	NOT NULL	订单总金额 (CNY)
currency	VARCHAR(10)	DEFAULT 'CNY'	支付货币
exchange_rate	DECIMAL(10,4)	NULL	下单时汇率 (海外)
address_id	BIGINT	FK addresses.id NULL	收货地址ID
address_snapshot	JSON	NULL	下单时地址快照
status	ENUM	DEFAULT 'pending_payment'	订单状态
payment_method	ENUM	NULL	支付方式
payment_id	BIGINT	FK payments.id NULL	支付记录ID
remark	VARCHAR(500)	NULL	买家备注
cancel_reason	VARCHAR(200)	NULL	取消原因
expires_at	DATETIME	NULL	支付超时时间
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

索引: `idx_order_no` (唯一)、`idx_user_id`、`idx_status`、`idx_expires_at`

订单状态流转：



1.11 order_items 订单明细表

描述： 订单下每个商品SKU的购买记录，含商品快照及定制信息。

```
CREATE TABLE `order_items` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '明细ID',  
  `order_id` BIGINT NOT NULL COMMENT '订单ID',  
  `product_id` BIGINT NOT NULL COMMENT '商品ID',  
  `sku_id` BIGINT NOT NULL COMMENT 'SKU ID',  
  `product_snapshot` JSON DEFAULT NULL COMMENT '下单时商品快照',  
  `sku_snapshot` JSON DEFAULT NULL COMMENT '下单时SKU快照',  
  `quantity` INT NOT NULL COMMENT '购买数量',  
  `unit_price` DECIMAL(10,2) NOT NULL COMMENT '下单时单价 (CNY) ',  
  `subtotal` DECIMAL(10,2) NOT NULL COMMENT '小计 (CNY) ',  
  `customization_id` BIGINT DEFAULT NULL COMMENT '刻字定制ID',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  KEY `idx_order_id` (`order_id`),  
  CONSTRAINT `fk_oi_order` FOREIGN KEY (`order_id`) REFERENCES `orders` (`id`),  
  CONSTRAINT `fk_oi_product` FOREIGN KEY (`product_id`) REFERENCES `products` (`id`),  
  CONSTRAINT `fk_oi_sku` FOREIGN KEY (`sku_id`) REFERENCES `product_skus` (`id`),  
  CONSTRAINT `fk_oi_custom` FOREIGN KEY (`customization_id`) REFERENCES `customizations` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='订单明细表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	明细ID
order_id	BIGINT	FK orders.id NOT NULL	订单ID
product_id	BIGINT	FK products.id NOT NULL	商品ID
sku_id	BIGINT	FK product_skus.id NOT NULL	SKU ID
product_snapshot	JSON	NULL	下单时商品快照
sku_snapshot	JSON	NULL	下单时SKU快照
quantity	INT	NOT NULL	购买数量
unit_price	DECIMAL(10,2)	NOT NULL	下单时单价
subtotal	DECIMAL(10,2)	NOT NULL	小计金额
customization_id	BIGINT	FK customizations.id NULL	刻字定制ID
created_at	DATETIME	NOT NULL	创建时间

1.12 customizations 刻字定制表

描述：用户对支持定制商品的刻字内容、字体及预览图记录。

```
CREATE TABLE `customizations` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '定制ID',  
  `user_id` BIGINT NOT NULL COMMENT '用户ID',  
  `product_id` BIGINT NOT NULL COMMENT '商品ID',  
  `text_content` VARCHAR(200) NOT NULL COMMENT '刻字文字内容（最多20字）',  
  `font_name` VARCHAR(50) DEFAULT NULL COMMENT '字体名称',  
  `preview_image_url` VARCHAR(500) DEFAULT NULL COMMENT '预览图URL (OSS) ',  
  `status` ENUM('draft','confirmed') NOT NULL DEFAULT 'draft' COMMENT '状态',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  KEY `idx_user_id` (`user_id`),  
  CONSTRAINT `fk_custom_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`),  
  CONSTRAINT `fk_custom_product` FOREIGN KEY (`product_id`) REFERENCES `products` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='刻字定制表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	定制ID
user_id	BIGINT	FK users.id NOT NULL	用户ID
product_id	BIGINT	FK products.id NOT NULL	商品ID
text_content	VARCHAR(200)	NOT NULL	刻字文字（最多20字）
font_name	VARCHAR(50)	NULL	字体名称
preview_image_url	VARCHAR(500)	NULL	预览图URL
status	ENUM	DEFAULT 'draft'	状态： draft/confirmed
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

1.13 payments 支付记录表

描述：记录每笔支付的完整信息，支持微信支付与PayPal双渠道。

```
CREATE TABLE `payments` (
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '支付记录ID',
  `payment_no` VARCHAR(64) NOT NULL COMMENT '内部支付单号',
  `order_id` BIGINT NOT NULL COMMENT '订单ID',
  `user_id` BIGINT NOT NULL COMMENT '用户ID',
  `amount` DECIMAL(10,2) NOT NULL COMMENT '支付金额 (本币) ',
  `currency` VARCHAR(10) NOT NULL COMMENT '支付货币',
  `exchange_rate` DECIMAL(10,4) DEFAULT NULL COMMENT '支付时汇率 (海外) ',
  `method` ENUM('wechat','paypal') NOT NULL COMMENT '支付方式',
  `status` ENUM('pending','success','failed','refunded') NOT NULL DEFAULT 'pending' COMMENT '支付状态',
  `wx_transaction_id` VARCHAR(64) DEFAULT NULL COMMENT '微信支付交易号',
  `wx_prepay_id` VARCHAR(64) DEFAULT NULL COMMENT '微信预支付ID',
  `paypal_order_id` VARCHAR(100) DEFAULT NULL COMMENT 'PayPal订单ID',
  `paypal_capture_id` VARCHAR(100) DEFAULT NULL COMMENT 'PayPal捕获ID',
  `paid_at` DATETIME DEFAULT NULL COMMENT '支付完成时间',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `idx_payment_no` (`payment_no`),
  KEY `idx_order_id` (`order_id`),
  KEY `idx_wx_transaction_id` (`wx_transaction_id`),
  KEY `idx_paypal_order_id` (`paypal_order_id`),
  CONSTRAINT `fk_pay_order` FOREIGN KEY (`order_id`) REFERENCES `orders` (`id`),
  CONSTRAINT `fk_pay_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='支付记录表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	支付记录ID
payment_no	VARCHAR(64)	UNIQUE NOT NULL	内部支付单号
order_id	BIGINT	FK orders.id NOT NULL	订单ID
user_id	BIGINT	FK users.id NOT NULL	用户ID
amount	DECIMAL(10,2)	NOT NULL	支付金额 (本币)
currency	VARCHAR(10)	NOT NULL	支付货币
exchange_rate	DECIMAL(10,4)	NULL	支付时汇率
method	ENUM	NOT NULL	支付方式: wechat/paypal
status	ENUM	DEFAULT 'pending'	支付状态
wx_transaction_id	VARCHAR(64)	NULL	微信支付交易号
wx_prepay_id	VARCHAR(64)	NULL	微信预支付ID
paypal_order_id	VARCHAR(100)	NULL	PayPal订单ID
paypal_capture_id	VARCHAR(100)	NULL	PayPal捕获ID
paid_at	DATETIME	NULL	支付完成时间
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

索引: idx_payment_no (唯一)、idx_order_id、idx_wx_transaction_id、idx_paypal_order_id

1.14 logistics 物流信息表

描述：存储订单的物流公司、运单号及物流轨迹信息。

```
CREATE TABLE `logistics` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '物流ID',  
  `order_id` BIGINT NOT NULL COMMENT '订单ID',  
  `company` VARCHAR(100) NOT NULL COMMENT '物流公司名称',  
  `tracking_no` VARCHAR(100) NOT NULL COMMENT '运单号',  
  `status` ENUM('pending','shipping','delivered') NOT NULL DEFAULT 'pending' COMMENT '物流状态',  
  `tracking_data` JSON DEFAULT NULL COMMENT '物流轨迹JSON数组',  
  `shipped_at` DATETIME DEFAULT NULL COMMENT '发货时间',  
  `delivered_at` DATETIME DEFAULT NULL COMMENT '签收时间',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `idx_order_id` (`order_id`),  
  KEY `idx_tracking_no` (`tracking_no`),  
  CONSTRAINT `fk_logistics_order` FOREIGN KEY (`order_id`) REFERENCES `orders` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='物流信息表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	物流ID
order_id	BIGINT	FK orders.id UNIQUE	订单ID（一单一物流）
company	VARCHAR(100)	NOT NULL	物流公司名称
tracking_no	VARCHAR(100)	NOT NULL	运单号
status	ENUM	DEFAULT 'pending'	物流状态
tracking_data	JSON	NULL	物流轨迹JSON
shipped_at	DATETIME	NULL	发货时间
delivered_at	DATETIME	NULL	签收时间
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

索引： idx_order_id （唯一）、 idx_tracking_no

1.15 after_sales 售后申请表

描述：用户申请退款、退货或换货的记录及审核结果。

```
CREATE TABLE `after_sales` (
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '售后ID',
  `after_sale_no` VARCHAR(32) NOT NULL COMMENT '售后编号',
  `order_id` BIGINT NOT NULL COMMENT '订单ID',
  `order_item_id` BIGINT DEFAULT NULL COMMENT '订单明细ID（部分退款时使用）',
  `user_id` BIGINT NOT NULL COMMENT '用户ID',
  `type` ENUM('refund','return','exchange') NOT NULL COMMENT '售后类型',
  `reason` VARCHAR(500) NOT NULL COMMENT '申请原因',
  `images` JSON DEFAULT NULL COMMENT '凭证图片URLs',
  `status` ENUM('pending','approved','rejected','completed') NOT NULL DEFAULT 'pending' COMMENT '审核状态',
  `reviewer_id` BIGINT DEFAULT NULL COMMENT '审核管理员ID',
  `review_remark` VARCHAR(500) DEFAULT NULL COMMENT '审核意见',
  `refund_amount` DECIMAL(10,2) DEFAULT NULL COMMENT '退款金额',
  `refund_at` DATETIME DEFAULT NULL COMMENT '退款完成时间',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `idx_after_sale_no` (`after_sale_no`),
  KEY `idx_order_id` (`order_id`),
  KEY `idx_user_id` (`user_id`),
  KEY `idx_status` (`status`),
  CONSTRAINT `fk_as_order` FOREIGN KEY (`order_id`) REFERENCES `orders` (`id`),
  CONSTRAINT `fk_as_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`),
  CONSTRAINT `fk_as_reviewer` FOREIGN KEY (`reviewer_id`) REFERENCES `admins` (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='售后申请表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	售后ID
after_sale_no	VARCHAR(32)	UNIQUE NOT NULL	售后编号
order_id	BIGINT	FK orders.id NOT NULL	订单ID
order_item_id	BIGINT	FK order_items.id NULL	明细ID（部分售后）
user_id	BIGINT	FK users.id NOT NULL	用户ID
type	ENUM	NOT NULL	售后类型：refund/return/exchange
reason	VARCHAR(500)	NOT NULL	申请原因
images	JSON	NULl	凭证图片URLs
status	ENUM	DEFAULT 'pending'	审核状态
reviewer_id	BIGINT	FK admins.id NULL	审核管理员ID
review_remark	VARCHAR(500)	NULl	审核意见
refund_amount	DECIMAL(10,2)	NULl	退款金额
refund_at	DATETIME	NULl	退款完成时间
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

索引： [idx_after_sale_no](#)（唯一）、[idx_order_id](#)、[idx_user_id](#)、[idx_status](#)

1.16 reviews 商品评价表

描述：用户对已完成订单中商品的评价内容与评分。

```
CREATE TABLE `reviews` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '评价ID',  
  `order_id` BIGINT NOT NULL COMMENT '订单ID',  
  `order_item_id` BIGINT NOT NULL COMMENT '订单明细ID',  
  `user_id` BIGINT NOT NULL COMMENT '用户ID',  
  `product_id` BIGINT NOT NULL COMMENT '商品ID',  
  `rating` TINYINT NOT NULL COMMENT '评分: 1-5星',  
  `content` VARCHAR(1000) DEFAULT NULL COMMENT '评价内容',  
  `is_anonymous` TINYINT(1) DEFAULT 0 COMMENT '是否匿名: 0否 1是',  
  `status` TINYINT DEFAULT 1 COMMENT '状态: 1正常 0隐藏',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uniq_order_item` (`order_item_id`),  
  KEY `idx_product_id` (`product_id`),  
  KEY `idx_user_id` (`user_id`),  
  KEY `idx_order_id` (`order_id`),  
  CONSTRAINT `fk_review_order` FOREIGN KEY (`order_id`) REFERENCES `orders` (`id`),  
  CONSTRAINT `fk_review_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`),  
  CONSTRAINT `fk_review_product` FOREIGN KEY (`product_id`) REFERENCES `products` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='商品评价表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	评价ID
order_id	BIGINT	FK orders.id NOT NULL	订单ID
order_item_id	BIGINT	FK order_items.id UNIQUE	订单明细ID（每明细只评一次）
user_id	BIGINT	FK users.id NOT NULL	用户ID
product_id	BIGINT	FK products.id NOT NULL	商品ID
rating	TINYINT	NOT NULL	评分：1-5星
content	VARCHAR(1000)	NULL	评价内容
is_anonymous	TINYINT(1)	DEFAULT 0	是否匿名
status	TINYINT	DEFAULT 1	状态：1正常 0隐藏
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

索引：idx_product_id 、 idx_user_id 、 idx_order_id ， UNIQUE(order_item_id)

1.17 review_images 评价图片表

描述：评价时上传的图片记录。

```
CREATE TABLE `review_images` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '图片ID',  
  `review_id` BIGINT NOT NULL COMMENT '评价ID',  
  `image_url` VARCHAR(500) NOT NULL COMMENT '图片URL',  
  `sort_order` INT DEFAULT 0 COMMENT '排序',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  KEY `idx_review_id` (`review_id`),  
  CONSTRAINT `fk_ri_review` FOREIGN KEY (`review_id`) REFERENCES `reviews` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='评价图片表';
```

1.18 notifications 消息通知表

描述：系统向用户发送的站内消息通知记录，含已读状态。

```
CREATE TABLE `notifications` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '通知ID',  
  `user_id` BIGINT NOT NULL COMMENT '用户ID',  
  `type` VARCHAR(50) NOT NULL COMMENT '通知类型 (如 order_paid, review_passed 等)',  
  `title` VARCHAR(200) NOT NULL COMMENT '通知标题',  
  `content` TEXT DEFAULT NULL COMMENT '通知内容',  
  `extra_data` JSON DEFAULT NULL COMMENT '扩展数据 (如跳转参数)',  
  `is_read` TINYINT(1) DEFAULT 0 COMMENT '是否已读: 0未读 1已读',  
  `read_at` DATETIME DEFAULT NULL COMMENT '阅读时间',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  KEY `idx_user_id` (`user_id`),  
  KEY `idx_is_read` (`is_read`),  
  CONSTRAINT `fk_notif_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='消息通知表';
```

字段名	类型	约束	说明
id	BIGINT	PK AUTO_INCREMENT	通知ID
user_id	BIGINT	FK users.id NOT NULL	用户ID
type	VARCHAR(50)	NOT NULL	通知类型
title	VARCHAR(200)	NOT NULL	通知标题
content	TEXT	NULL	通知内容
extra_data	JSON	NULL	扩展数据
is_read	TINYINT(1)	DEFAULT 0	是否已读
read_at	DATETIME	NULL	阅读时间
created_at	DATETIME	NOT NULL	创建时间

索引： idx_user_id 、 idx_is_read

1.19 admins 管理员表

描述：后台管理系统的管理员账号信息。

```
CREATE TABLE `admins` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '管理员ID',  
  `username` VARCHAR(50) NOT NULL COMMENT '登录用户名',  
  `password_hash` VARCHAR(255) NOT NULL COMMENT '密码哈希 (bcrypt) ',  
  `real_name` VARCHAR(100) DEFAULT NULL COMMENT '真实姓名',  
  `email` VARCHAR(200) DEFAULT NULL COMMENT '邮箱',  
  `phone` VARCHAR(20) DEFAULT NULL COMMENT '手机号',  
  `last_login_at` DATETIME DEFAULT NULL COMMENT '最后登录时间',  
  `status` TINYINT DEFAULT 1 COMMENT '状态: 1启用 0禁用',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `idx_username` (`username`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='管理员表';
```

1.20 roles 角色表

描述：RBAC权限模型中的角色定义。

```
CREATE TABLE `roles` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '角色ID',  
  `name` VARCHAR(100) NOT NULL COMMENT '角色标识 (英文唯一键) ',  
  `display_name` VARCHAR(200) DEFAULT NULL COMMENT '角色显示名称',  
  `description` VARCHAR(500) DEFAULT NULL COMMENT '角色描述',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `idx_name` (`name`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='角色表';
```

1.21 permissions 权限表

描述：系统功能权限项，按模块分组管理。

```
CREATE TABLE `permissions` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '权限ID',  
  `name` VARCHAR(100) NOT NULL COMMENT '权限标识 (如 order:manage) ',  
  `display_name` VARCHAR(200) DEFAULT NULL COMMENT '权限显示名称',  
  `module` VARCHAR(50) DEFAULT NULL COMMENT '所属模块',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `idx_name` (`name`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='权限表';
```

1.22 role_permissions 角色权限关联表

描述：多对多关联角色与权限。

```
CREATE TABLE `role_permissions` (  
  `role_id` BIGINT NOT NULL COMMENT '角色ID',  
  `permission_id` BIGINT NOT NULL COMMENT '权限ID',  
  PRIMARY KEY (`role_id`, `permission_id`),  
  CONSTRAINT `fk_rp_role` FOREIGN KEY (`role_id`) REFERENCES `roles` (`id`),  
  CONSTRAINT `fk_rp_perm` FOREIGN KEY (`permission_id`) REFERENCES `permissions` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='角色权限关联表';
```

1.23 admin_roles 管理员角色关联表

描述：多对多关联管理员与角色。

```
CREATE TABLE `admin_roles` (  
  `admin_id` BIGINT NOT NULL COMMENT '管理员ID',  
  `role_id` BIGINT NOT NULL COMMENT '角色ID',  
  PRIMARY KEY (`admin_id`, `role_id`),  
  CONSTRAINT `fk_ar_admin` FOREIGN KEY (`admin_id`) REFERENCES `admins` (`id`),  
  CONSTRAINT `fk_ar_role` FOREIGN KEY (`role_id`) REFERENCES `roles` (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='管理员角色关联表';
```

1.24 banners Banner配置表

描述：首页轮播图/Banner的内容与跳转配置。

```
CREATE TABLE `banners` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT 'Banner ID',  
  `title` VARCHAR(200) DEFAULT NULL COMMENT 'Banner标题',  
  `image_url` VARCHAR(500) NOT NULL COMMENT '图片URL',  
  `link_type` ENUM('product','url','none') NOT NULL DEFAULT 'none' COMMENT '跳转类型',  
  `link_value` VARCHAR(500) DEFAULT NULL COMMENT '跳转值（商品ID或外链URL）',  
  `sort_order` INT DEFAULT 0 COMMENT '排序权重',  
  `status` TINYINT DEFAULT 1 COMMENT '状态：1启用 0禁用',  
  `start_at` DATETIME DEFAULT NULL COMMENT '生效开始时间',  
  `end_at` DATETIME DEFAULT NULL COMMENT '生效结束时间',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='Banner配置表';
```

1.25 announcements 公告表

描述：校庆相关公告与活动信息发布。

```
CREATE TABLE `announcements` (
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '公告ID',
  `title` VARCHAR(200) NOT NULL COMMENT '公告标题',
  `content` TEXT NOT NULL COMMENT '公告内容',
  `type` ENUM('normal','important','urgent') NOT NULL DEFAULT 'normal' COMMENT '公告类型',
  `status` TINYINT DEFAULT 1 COMMENT '状态: 1发布 0草稿',
  `published_at` DATETIME DEFAULT NULL COMMENT '发布时间',
  `expires_at` DATETIME DEFAULT NULL COMMENT '到期时间',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='公告表';
```

1.26 system_settings 系统配置表

描述：键值对形式存储全局系统配置参数。

```
CREATE TABLE `system_settings` (
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '配置ID',
  `key` VARCHAR(100) NOT NULL COMMENT '配置键',
  `value` TEXT DEFAULT NULL COMMENT '配置值',
  `description` VARCHAR(500) DEFAULT NULL COMMENT '配置说明',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `idx_key` (`key`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='系统配置表';
```

内置配置键示例：

key	说明
order_expire_minutes	订单支付超时时间（分钟），默认30
customization_max_chars	刻字最大字符数，默认20
overseas_shipping_enabled	是否开启海外配送
paypal_enabled	是否启用PayPal支付
review_auto_approve_days	订单完成后自动好评天数

1.27 exchange_rates 汇率记录表

描述：记录每次获取的汇率数据，供海外订单换算使用。

```
CREATE TABLE `exchange_rates` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '汇率记录ID',  
  `from_currency` VARCHAR(10) NOT NULL COMMENT '源货币 (如 USD) ',  
  `to_currency` VARCHAR(10) NOT NULL COMMENT '目标货币 (如 CNY) ',  
  `rate` DECIMAL(10,6) NOT NULL COMMENT '汇率值',  
  `source` VARCHAR(50) NOT NULL COMMENT '来源: manual/api',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  KEY `idx_currencies_created` (`from_currency`, `to_currency`, `created_at`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='汇率记录表';
```

1.28 report_schedules 报表邮件计划表

描述：定期报表邮件发送计划配置。

```
CREATE TABLE `report_schedules` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '计划ID',  
  `name` VARCHAR(200) NOT NULL COMMENT '计划名称',  
  `type` ENUM('daily','weekly','monthly') NOT NULL COMMENT '发送频率',  
  `recipients` JSON NOT NULL COMMENT '收件人邮箱列表',  
  `report_types` JSON NOT NULL COMMENT '报表类型列表',  
  `is_enabled` TINYINT(1) DEFAULT 1 COMMENT '是否启用',  
  `last_sent_at` DATETIME DEFAULT NULL COMMENT '上次发送时间',  
  `next_send_at` DATETIME NOT NULL COMMENT '下次发送时间',  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='报表邮件计划表';
```

2. 核心业务流程详细设计

2.1 认证流程（REQ-001～REQ-010）

2.1.1 微信登录流程



详细步骤说明：

1. 用户点击登录，小程序调用 `wx.login()` 获取临时 code（有效期5分钟）
2. 小程序将 code POST 至 `/api/auth/wx-login`
3. 后端使用 AppID + AppSecret + code 请求微信 code2session 接口，获取 openid 和 session_key
4. 后端查询 `users` 表：
 - 若 openid 已存在：返回现有用户信息
 - 若 openid 不存在：创建新用户记录（status=1）
5. 生成 JWT Token（payload含user_id、openid，有效期7天）
6. 返回 token 及用户基本信息给小程序
7. 小程序将 token 存入 storage，后续请求在 Header 中携带 `Authorization: Bearer {token}`

2.1.2 校友认证流程

- 用户 → 填写认证信息（姓名/学号/毕业年份/专业）
- 上传证明材料图片（POST /upload → OSS）
 - 提交认证申请（POST /auth/alumni/apply）
 - 系统写入 `alumni_verifications`（status=pending）
- 管理员 → 后台查看待审核列表（GET /admin/alumni/verifications?status=pending）
- 查看申请详情（包含证明图片）
 - 执行审核操作（PUT /admin/alumni/verifications/{id}/review）
 - [通过] → `alumni_verifications.status=approved`
`users.is_verified=1`
发送微信订阅消息：审核通过通知
 - [拒绝] → `alumni_verifications.status=rejected`
填写拒绝原因（review_remark）
发送微信订阅消息：审核未通过通知（含原因）

预留校友系统API自动验证：

当管理员配置了外部校友数据库API后，系统在用户提交认证时，自动调用外部API传入学号进行验证。若API返回已认证，直接设置 `status=approved`，无需人工审核。

数据流：

步骤	操作	涉及表
用户提交	写入认证申请	alumni_verifications
图片上传	保存OSS路径	alumni_verifications.proof_image_url
审核通过	更新认证状态	alumni_verifications, users
消息通知	写入通知记录	notifications

2.2 国内购买 + 微信支付流程（REQ-031~REQ-047）

[下单阶段]

用户选择SKU → 加入购物车 or 立即购买

- 确认收货地址
- 提交订单 POST /api/orders
 - ├─ 验证用户已完成校友认证 (is_verified=1)
 - ├─ 验证库存 (Redis原子检查)
 - ├─ 扣减库存 (Redis DECRBY + MySQL乐观锁)
 - ├─ 创建 orders 记录 (status=pending_payment)
 - ├─ 创建 order_items 记录 (含商品快照)
 - ├─ 保存 address_snapshot (地址快照)
 - ├─ 设置 expires_at = NOW() + 30min
 - └─ ZADD order:timeout {timestamp} {order_id} (延迟队列)

[支付阶段]

POST /api/payments/wx/prepay

- 调用微信支付统一下单API (/v3/pay/transactions/jsapi)
- 获取 prepay_id, 写入 payments 表 (status=pending)
- 返回小程序支付参数 (timeStamp, nonceStr, package, signType, paySign)

小程序 wx.requestPayment() → 用户输入支付密码 → 支付完成

[回调阶段]

微信回调 POST /api/payments/wx/notify

- 验证签名 (微信APIv3签名)
- 解密通知数据 (AES-256-GCM)
- 更新 payments (status=success, wx_transaction_id, paid_at)
- 更新 orders (status=paid)
- ZREM order:timeout {order_id} (从超时队列移除)
- 发送"支付成功"订阅消息
- 返回 HTTP 200 {"code":"SUCCESS"}

[超时处理]

定时任务 (每30秒执行):

ZRANGEBYSCORE order:timeout 0 {NOW_TIMESTAMP}

- 对每个到期订单:
 - ├─ 若 orders.status=pending_payment
 - orders.status=cancelled, cancel_reason='支付超时自动关闭'
 - 回滚库存 (Redis INCRBY + MySQL stock+quantity)
 - ZREM order:timeout {order_id}

接口说明:

接口	方法	说明
/api/orders	POST	创建订单
/api/payments/wx/prepay	POST	发起微信支付预下单
/api/payments/wx/notify	POST	微信支付回调（不需鉴权）
/api/orders/{id}/cancel	POST	用户主动取消订单

2.3 海外购买 + PayPal流程 (REQ-041~REQ-048)

[汇率获取]

GET /api/payments/exchange-rate

- 优先从 Redis 获取 (key: exchange_rate:USD:CNY, TTL 1小时)
- Redis 未命中 → 查 exchange_rates 表最新记录
- 前端展示: $USD = CNY \div rate$ (保留2位小数)

[下单阶段]

POST /api/orders

- 与国内流程相同, 但:
 - └ currency = 'USD'
 - └ exchange_rate = 当前汇率 (记录快照)

[PayPal支付]

POST /api/payments/paypal/create

- 调用 PayPal Orders API v2 (POST /v2/checkout/orders)
- intent: CAPTURE
- amount: { value: "XX.XX", currency_code: "USD" }
- 返回 PayPal order_id 及 approve_url
- 写入 payments 表 (status=pending, paypal_order_id)

小程序内嵌 WebView 打开 approve_url (PayPal收银台)

用户完成付款 → PayPal 跳转回小程序 return_url

POST /api/payments/paypal/capture/{paypal_order_id}

- 调用 PayPal Capture API (POST /v2/checkout/orders/{id}/capture)
- 获取 capture_id
- 更新 payments (status=success, paypal_capture_id, paid_at)
- 更新 orders (status=paid)
- 发送"支付成功"通知

[PayPal Webhook 备份验证]

POST /api/payments/paypal/webhook

- 验证 Webhook 签名 (PayPal-Transmission-Sig)
- 处理 PAYMENT.CAPTURE.COMPLETED 事件
- 作为支付确认的幂等备份

汇率定时任务:

每小时执行:

1. 调用汇率API (如 fixer.io / 开放汇率API) 获取 USD/CNY
2. 写入 exchange_rates 表 (source='api')
3. SET exchange_rate:USD:CNY {rate} EX 3600
4. 管理员手动设置时: SET exchange_rate:USD:CNY {rate} EX 86400 (优先级最高, TTL更长)

2.4 定制刻字下单流程（REQ-059～REQ-065）

[刻字配置]

用户进入商品详情页

└─ 若 products.is_customizable=1, 显示"添加刻字"入口

点击"添加刻字" → 进入定制页面

- └─ 输入文字 (≤20字, 支持汉字/英文/数字)
- └─ POST /api/customization/validate
 - 后端校验: 字数、敏感词过滤 (接入内容安全API)
 - 返回验证结果
- └─ GET /api/customization/fonts → 获取可用字体列表
- └─ 选择字体
- └─ POST /api/customization/preview
 - 后端调用图片处理服务 (如 Python Pillow / ImageMagick)
 - 将文字渲染到商品模板图片上
 - 生成预览图上传至 OSS
 - 返回预览图 URL

用户确认预览

- POST /api/customization/confirm
- 写入 customizations 表 (status=confirmed)
- 返回 customization_id

加入购物车 / 立即购买

- 携带 customization_id
- cart_items.customization_text 冗余存储文字内容
- 创建订单时: order_items.customization_id = customization_id

[管理员查看]

订单详情页显示刻字要求:

- customizations.text_content (文字内容)
- customizations.font_name (字体)
- customizations.preview_image_url (预览图)

刻字预览图生成服务技术说明:

- 使用 Python + Pillow 库实现文字渲染
- 每种商品维护一张模板底图 (透明PNG), 定义文字渲染区域的坐标和尺寸
- 输出 JPEG, 分辨率 800×800px
- 生成时间 < 2秒, 预览图有效期7天 (OSS生命周期策略自动清理草稿)

2.5 售后退款流程 (REQ-054~REQ-058, REQ-113~REQ-120)

[用户申请]

用户在订单详情 (status=completed 或 shipped) 点击"申请售后"

- 选择售后类型: 退款 / 退货 / 换货
- 填写原因, 上传凭证图片 (最多5张)
- POST /api/after-sales
 - 创建 after_sales 记录 (status=pending)
 - 更新 orders.status=refunding

[管理员审核]

后台售后列表 (GET /admin/after-sales?status=pending)

- 查看申请详情 (含凭证图片、原始订单信息)
- 执行审核: PUT /admin/after-sales/{id}/review

[审核通过]

- after_sales.status=approved
- 根据支付方式执行退款:
 - └─ 微信支付: POST 微信退款API (/v3/refund/domestic/refunds)
 - 返回 refund_id
 - 微信异步回调 /api/payments/wx/refund-notify
 - 更新 payments.status=refunded
 - 更新 after_sales.status=completed, refund_at
 - 更新 orders.status=refunded
 - 发送"退款成功"通知
 - └─ PayPal: POST /v2/payments/captures/{capture_id}/refund
 - 获取 refund_id
 - 同步更新 payments、after_sales、orders 状态
 - 发送"退款成功"通知

[审核拒绝]

- after_sales.status=rejected
- 填写 review_remark (拒绝原因)
- 恢复 orders.status=completed (回滚到原状态)
- 发送"售后申请已拒绝"通知 (含拒绝原因)

3. 关键算法设计

3.1 库存原子扣减

设计目标: 高并发场景下防止超卖, 保证库存数据一致性。

两阶段扣减方案:

阶段一：Redis 原子操作（快速拒绝超卖）

```
SCRIPT:
  local stock = redis.call('GET', 'product:stock:' .. sku_id)
  if tonumber(stock) < quantity then
    return -1  -- 库存不足
  end
  return redis.call('DECRBY', 'product:stock:' .. sku_id, quantity)
```

若返回值 < 0:

```
INCRBY product:stock:{sku_id} quantity  -- 回滚 Redis
返回"库存不足"错误
```

阶段二：MySQL 乐观锁兜底（数据持久化）

```
UPDATE product_skus
SET stock = stock - {quantity}
WHERE id = {sku_id}
AND stock >= {quantity}
```

若受影响行数 = 0:

```
INCRBY product:stock:{sku_id} quantity  -- 回滚 Redis
返回"库存不足"错误
```

若成功:

同步更新 products.stock (冗余字段)

库存初始化（服务启动时）:

```
# 从 MySQL 同步库存到 Redis
for sku in product_skus.query(status=1):
    redis.set(f'product:stock:{sku.id}', sku.stock)
```

库存回滚（订单取消/超时时）:

```
# 1. Redis 回滚
redis.incrby(f'product:stock:{sku_id}', quantity)

# 2. MySQL 回滚
UPDATE product_skus SET stock = stock + {quantity} WHERE id = {sku_id}
```

3.2 订单超时自动关闭

实现方案：Redis Sorted Set 延迟队列 + 定时轮询

```

# 创建订单时: 将 expires_at 时间戳作为 score 写入 Sorted Set
expires_timestamp = int((datetime.now() + timedelta(minutes=30)).timestamp())
redis.zadd('order:timeout', {order_id: expires_timestamp})

# 定时任务 (每30秒):
def check_expired_orders():
    now = int(datetime.now().timestamp())
    # 获取所有已到期的订单ID
    expired_ids = redis.zrangebyscore('order:timeout', 0, now)

    for order_id in expired_ids:
        # 查询当前订单状态 (避免重复处理)
        order = Order.get(order_id)
        if order and order.status == 'pending_payment':
            with db.transaction():
                # 关闭订单
                order.update(status='cancelled', cancel_reason='支付超时自动关闭')
                # 回滚库存
                for item in order.items:
                    rollback_stock(item.sku_id, item.quantity)
            # 从队列移除
            redis.zrem('order:timeout', order_id)

```

幂等保障: 定时任务使用分布式锁 (Redis SETNX), 防止多实例并发执行。

3.3 汇率换算算法

汇率获取优先级: 手动设置 > Redis缓存 > 数据库最新记录

```

def get_exchange_rate(from_currency: str, to_currency: str) -> Decimal:
    key = f'exchange_rate:{from_currency}:{to_currency}'

    # 1. 从 Redis 获取 (TTL 1小时, 手动设置时 TTL 24小时)
    rate = redis.get(key)
    if rate:
        return Decimal(rate)

    # 2. 从数据库获取最新记录
    record = ExchangeRate.query(
        from_currency=from_currency,
        to_currency=to_currency
    ).order_by('-created_at').first()

    if record:
        redis.setex(key, 3600, str(record.rate))
        return record.rate

    raise Exception('汇率数据不可用')

def calculate_usd_price(cny_price: Decimal) -> Decimal:
    rate = get_exchange_rate('USD', 'CNY')
    usd_price = cnr_price / rate
    return usd_price.quantize(Decimal('0.01'), rounding=ROUND_HALF_UP)

```

定时同步任务（每小时）：

```
def sync_exchange_rate():
    response = requests.get('https://api.exchangerate-api.com/v4/latest/USD')
    rate = response.json()['rates']['CNY']

    # 写入数据库
    ExchangeRate.create(from_currency='USD', to_currency='CNY',
                        rate=rate, source='api')

    # 更新 Redis（若无手动覆盖则TTL=1小时）
    key = 'exchange_rate:USD:CNY'
    if not redis.exists('exchange_rate:USD:CNY:manual'):
        redis.setex(key, 3600, str(rate))
```

3.4 订单编号生成算法

格式：XQ{YYYYMMDD}-{6位随机数}，如 XQ2026030900001

```
import time
import random

def generate_order_no() -> str:
    date_str = time.strftime('%Y%m%d')
    # Redis 原子自增，确保当天唯一
    seq = redis.incr(f'order:seq:{date_str}')
    redis.expire(f'order:seq:{date_str}', 86400)
    return f'XQ{date_str}{str(seq).zfill(6)}'
```

3.5 刻字预览图生成算法

```
from PIL import Image, ImageDraw, ImageFont
import io

def generate_preview(product_id: int, text: str, font_name: str) -> bytes:
    # 加载商品模板图（透明背景）
    template = Image.open(f'templates/{product_id}.png').convert('RGBA')

    # 加载字体
    font_path = f'fonts/{font_name}.ttf'
    font_size = calculate_font_size(text, template.width * 0.6)
    font = ImageFont.truetype(font_path, size=font_size)

    # 计算文字居中坐标
    draw = ImageDraw.Draw(template)
    bbox = draw.textbbox((0, 0), text, font=font)
    text_w = bbox[2] - bbox[0]
    text_h = bbox[3] - bbox[1]
    x = (template.width - text_w) / 2
    y = template.height * 0.75 # 文字区域在商品下方75%处

    # 渲染文字（金色 #D4AF37）
    draw.text((x, y), text, font=font, fill='#D4AF37')

    # 转换为 JPEG 输出
    output = io.BytesIO()
    template.convert('RGB').save(output, format='JPEG', quality=90)
    return output.getvalue()
```

4. 需求对照表

全量覆盖 REQ-001 至 REQ-145，每条需求对应实现方案。

需求编号	需求名称	对应表/模块	实现细节
REQ-001	微信一键登录	users	调用 wx.login() 获取code，后端 code2session 换取 openid，创建/查询 users 记录，返回 JWT Token
REQ-002	用户信息获取	users	引导用户授权获取 nickname/avatar_url，写入 users 表；手机号需用 getPhoneNumber 按钮获取
REQ-003	手机号绑定	users	调用微信 getPhoneNumber 接口，获取加密手机号，后端解密写入 users.phone + country_code
REQ-004	海外用户识别	users	注册时选择区号，若非+86则设 is_overseas=1；登录后引导填写国际地址
REQ-005	校友认证申请	alumni_verifications	用户填写姓名/学号/毕业年份/专业，上传证明图片，POST /auth/alumni/apply，写入 pending 记录
REQ-006	认证状态查询	alumni_verifications	GET /auth/alumni/status 返回最新申请记录状态及审核意见

需求编号	需求名称	对应表/模块	实现细节
REQ-007	认证结果通知	notifications	审核完成后写入 notifications 表，同时发送微信订阅消息模板
REQ-008	外部校友系统对接（预留）	alumni_verifications	预留接口，通过学号调用外部API自动核验，验证通过直接 status=approved
REQ-009	用户登出	-	前端清除本地 storage 中的 JWT Token，后端可选择将 token 加入黑名单（Redis TTL）
REQ-010	认证权限控制	users.is_verified	购物、下单等接口通过中间件校验 users.is_verified=1，未认证返回 403
REQ-011	首页 Banner 轮播	banners	GET /api/home/banners 返回 status=1 且在有效期内的 banner 列表，按 sort_order 排序
REQ-012	首页公告展示	announcements	GET /api/home/announcements 返回已发布且未过期的公告，type=urgent 置顶
REQ-013	首页商品推荐	products	GET /api/home/products 返回 status=published 商品，按 sort_order 倒序取 TOP N
REQ-014	首页分类导航	product_categories	GET /api/home/categories 返回一级分类列表（parent_id IS NULL）及图标
REQ-015	首页搜索入口	products	点击搜索跳转商品搜索页，支持商品名称全文检索（MySQL LIKE 或 Elasticsearch）
REQ-016	商品列表展示	products, product_skus	GET /api/products?category_id=&keyword=&page=&size= 分页返回已发布商品
REQ-017	商品分类筛选	product_categories, products	按 category_id 过滤，支持二级分类级联查询（查父分类及所有子分类ID）
REQ-018	商品详情展示	products, product_images, product_skus	GET /api/products/{id} 返回商品详情、图片列表、SKU规格及库存
REQ-019	SKU 规格选择	product_skus	前端根据 spec_values JSON 渲染规格选择器，选中 SKU 后显示对应价格/库存/图片
REQ-020	商品库存显示	product_skus	从 Redis 实时读取 product:stock:{sku_id}，stock=0 时显示"已售罄"并禁用购买按钮
REQ-021	商品价格展示	products, product_skus, exchange_rates	国内用户显示 CNY 价格；海外用户同时显示 USD 换算价格（实时汇率）
REQ-022	商品搜索	products	GET /api/products/search?keyword= 对 name/name_en/description 进行模糊匹配
REQ-023	商品收藏（预留）	-	预留 user_favorites 表，本期暂不实现
REQ-024	商品分享	products	小程序内置 onShareAppMessage 生成商品详情分享卡片（含封面图、名称、价格）
REQ-025	商品评价汇总展示	reviews	商品详情页底部展示评分均值、评价总数及最新3条评价

需求编号	需求名称	对应表/模块	实现细节
REQ-026	加入购物车	carts, cart_items	POST /api/cart/items, 若用户无购物车先创建 carts 记录, 再 INSERT/UPDATE cart_items
REQ-027	购物车商品列表	cart_items, products, product_skus	GET /api/cart 返回 cart_items 关联商品/SKU的最新信息, 实时校验库存可用性
REQ-028	购物车数量修改	cart_items	PUT /api/cart/items/{id}, 更新 quantity, 校验不超过当前库存
REQ-029	购物车商品删除	cart_items	DELETE /api/cart/items/{id}, 支持批量删除
REQ-030	购物车选中结算	cart_items	前端维护选中状态, 结算时传入选中的 cart_item_ids; 不在数据库持久化选中状态
REQ-031	订单创建	orders, order_items	POST /api/orders, 验证认证状态、库存扣减、 快照保存、超时队列入队
REQ-032	收货地址选择	addresses	下单前 GET /api/addresses 获取用户地址列表, 选择后传 address_id; 下单时保存 address_snapshot
REQ-033	订单商品备注	orders.remark	下单时可填写 remark 字段, 管理员在订单详情可见
REQ-034	订单列表查询	orders, order_items	GET /api/orders?status=&page= 返回用户订单列表, 支持按状态筛选
REQ-035	订单详情查看	orders, order_items, logistics	GET /api/orders/{id} 返回订单完整信息含物流状态
REQ-036	订单取消	orders	POST /api/orders/{id}/cancel, 限 status=pending_payment 时可取消, 回滚库存
REQ-037	订单确认收货	orders	POST /api/orders/{id}/confirm-receipt, status=shipped → completed, 触发好评提醒
REQ-038	订单超时自动取消	orders	Redis Sorted Set 延迟队列, 定时任务每30秒扫描过期订单并关闭 (详见3.2节)
REQ-039	订单状态通知	notifications	订单每次状态变更 (付款成功/已发货/已完成/已取消) 均写 notifications 并发订阅消息
REQ-040	直接购买 (不经购物车)	orders	商品详情页"立即购买"直接传 product_id/sku_id/quantity/customization_id 创建订单
REQ-041	微信支付发起	payments	POST /api/payments/wx/prepay, 调用微信 JSAPI 支付统一下单, 返回小程序支付参数
REQ-042	微信支付回调	payments, orders	POST /api/payments/wx/notify, 验签→解密→ 更新支付状态→更新订单状态→通知用户
REQ-043	PayPal 支付发起	payments	POST /api/payments/paypal/create, 调用 PayPal Orders API v2 创建订单, 返回 approve_url
REQ-044	PayPal 支付捕获	payments, orders	POST /api/payments/paypal/capture/{paypal_order_id}, 调用 Capture API 完成扣款

需求编号	需求名称	对应表/模块	实现细节
REQ-045	PayPal Webhook 回调	payments	POST /api/payments/paypal/webhook, 验证签名, 处理 PAYMENT.CAPTURE.COMPLETED 事件
REQ-046	汇率实时换算	exchange_rates	GET /api/payments/exchange-rate, 从 Redis 获取最新 USD/CNY 汇率, 前端计算展示
REQ-047	支付结果页展示	payments, orders	支付完成后跳转支付结果页, 轮询或通过 WebSocket 获取支付结果 (状态轮询间隔2秒, 最多15次)
REQ-048	支付超时提醒	orders	下单后在小程序倒计时展示剩余支付时间 (expires_at - now), 到期提示订单已关闭
REQ-049	物流信息查看	logistics	GET /api/orders/{id}/logistics 返回物流公司、运单号及轨迹 tracking_data
REQ-050	物流轨迹展示	logistics.tracking_data	tracking_data 为 JSON 数组, 按时间倒序展示每个轨迹节点 (时间、地点、状态)
REQ-051	物流状态推送	notifications	物流状态变更 (已发货/已签收) 时发送微信订阅消息通知用户
REQ-052	国际物流支持	logistics	物流公司字段支持 DHL、FedEx 等国际快递, tracking_data 格式统一
REQ-053	签收确认	orders, logistics	用户主动确认收货 或 物流状态=delivered 后自动触发确认 (系统延迟7天自动收货)
REQ-054	售后申请提交	after_sales	POST /api/after-sales, 支持退款/退货/换货三种类型, 上传凭证图片到 OSS
REQ-055	售后类型选择	after_sales.type	枚举类型: refund (仅退款)、return (退货退款)、exchange (换货)
REQ-056	售后状态跟踪	after_sales	GET /api/after-sales/{id} 查看售后进度, 状态: pending→approved/rejected→completed
REQ-057	退款到账	payments, after_sales	微信/PayPal 退款API调用成功后, 更新 payments.status=refunded, after_sales.status=completed
REQ-058	售后结果通知	notifications	售后审核通过或拒绝后发送通知, 拒绝时包含 review_remark 原因
REQ-059	刻字商品标识	products.is_customizable	商品列表/详情展示"支持刻字"标签, is_customizable=1 时显示
REQ-060	刻字内容输入	customizations.text_content	最多20字, 支持中英文数字, 输入框实时计数, 超出20字禁止输入
REQ-061	刻字内容审核	customizations	POST /api/customization/validate, 调用微信内容安全 API (msgSecCheck) 过滤违禁词
REQ-062	字体选择	customizations.font_name	GET /api/customization/fonts 返回可用字体列表 (含字体预览样本图), 用户选择后记录 font_name
REQ-063	刻字预览图生成	customizations.preview_image_url	POST /api/customization/preview, 后端 Pillow 渲染预览图上传至 OSS, 返回图片 URL

需求编号	需求名称	对应表/模块	实现细节
REQ-064	刻字定制确认	customizations.status	POST /api/customization/confirm, status: draft→confirmed, 绑定 customization_id
REQ-065	刻字信息随订单保存	order_items.customization_id	下单时 order_items 关联 customization_id, 管理员在订单详情可查看完整刻字要求及预览图
REQ-066	评价入口展示	reviews	订单状态=completed 且无评价记录时, 在订单详情展示"去评价"按钮
REQ-067	星级评分	reviews.rating	1-5星评分, 提交时校验范围; 商品详情展示平均分 (SUM/COUNT)
REQ-068	图文评价	reviews, review_images	支持文字评价 (≤1000字) + 最多9张图片, 图片上传至 OSS
REQ-069	匿名评价	reviews.is_anonymous	用户选择匿名时, 前端展示"匿名用户"替代昵称, 头像显示默认图
REQ-070	评价列表展示	reviews, review_images	商品详情页评价区, 按 created_at 倒序分页展示, 每页10条
REQ-071	评价点赞 (预留)	-	预留 review_likes 表, 本期暂不实现
REQ-072	订单完成自动提醒评价	notifications	订单状态变为 completed 时, 延迟24小时发送评价提醒通知 (定时任务)
REQ-073	订单状态消息通知	notifications	支付成功/待发货/已发货/确认收货 状态变更时, 写 notifications 并调用微信订阅消息API
REQ-074	校友认证结果通知	notifications	认证审核完成后, 写 notifications (type=alumni_review_result) 并发订阅消息
REQ-075	售后进度通知	notifications	售后状态变更时通知用户 (审核中/已通过/已拒绝/退款成功)
REQ-076	站内消息列表	notifications	GET /api/notifications?page= 返回用户通知列表, is_read=0 的通知展示红点
REQ-077	消息已读标记	notifications	PUT /api/notifications/{id}/read 或 PUT /api/notifications/read-all 标记已读
REQ-078	微信订阅消息发送	notifications	调用微信 subscribeMessage.send API, 需用户事先订阅相应模板; 失败时记录错误日志不影响主流程
REQ-079	管理员登录	admins	POST /admin/auth/login, 校验 username+password (bcrypt 比对), 返回后台 JWT Token
REQ-080	管理员账号管理	admins	CRUD 管理员账号, 支持启用/禁用 (status 字段), 密码 bcrypt 加密存储
REQ-081	角色管理	roles	创建/编辑/删除角色, 为角色分配权限 (role_permissions 关联)
REQ-082	权限管理	permissions	预置系统权限项 (如 order:manage, product:manage), 按模块分组展示

需求编号	需求名称	对应表/模块	实现细节
REQ-083	管理员角色分配	admin_roles	为管理员分配一个或多个角色（admin_roles 多对多关联）
REQ-084	权限校验中间件	admins, admin_roles, role_permissions	后台接口通过 JWT 解析 admin_id，查询其权限集合，不满足则返回 403
REQ-085	操作日志记录	admin_logs（新增扩展表）	管理员关键操作（审核、修改配置、退款等）记录到 admin_logs 表，含操作人/时间/内容
REQ-086	校友认证列表	alumni_verifications	GET /admin/alumni/verifications?status=&keyword=&page= 支持状态筛选、姓名/学号搜索
REQ-087	校友认证详情	alumni_verifications	GET /admin/alumni/verifications/{id} 展示申请详情含证明图片
REQ-088	校友认证审核	alumni_verifications, users	PUT /admin/alumni/verifications/{id}/review, 审核通过则同步更新 users.is_verified
REQ-089	批量校友导入	users, alumni_verifications	POST /admin/alumni/import（CSV文件），批量创建已认证校友账号（status=approved）
REQ-090	用户列表管理	users	GET /admin/users?keyword=&is_verified=&page= 查询所有用户，支持禁用账号
REQ-091	用户详情查看	users, alumni_verifications, orders	GET /admin/users/{id} 查看用户基本信息、认证状态及历史订单
REQ-092	用户账号封禁	users.status	PUT /admin/users/{id}/status, 设置 status=0 封禁后该用户 token 失效（黑名单机制）
REQ-093	商品新增	products, product_skus, product_images	POST /admin/products, 支持设置基本信息、图片、SKU规格及库存
REQ-094	商品编辑	products, product_skus	PUT /admin/products/{id}, 编辑已有商品信息；修改 SKU 需同步更新 Redis 库存缓存
REQ-095	商品上架/下架	products.status	PUT /admin/products/{id}/status, 切换 published/unpublished；下架商品不影响已有订单
REQ-096	商品分类管理	product_categories	CRUD 商品分类，支持拖拽排序（sort_order 字段），最多二级层级
REQ-097	商品图片管理	product_images	上传/删除商品图片，支持拖拽调整排序（sort_order 字段）
REQ-098	SKU 库存调整	product_skus	PUT /admin/products/{id}/skus/{sku_id}/stock, 手动调整库存，同步更新 Redis
REQ-099	商品列表查看	products	GET /admin/products?status=&category_id=&keyword=&page= 分页查询，支持多维度筛选
REQ-100	商品详情查看	products, product_skus, product_images	GET /admin/products/{id} 展示完整商品信息
REQ-101	刻字字体管理	customization_fonts（扩展表）	CRUD 字体配置，上传字体文件至 OSS，管理可用字体列表

需求编号	需求名称	对应表/模块	实现细节
REQ-102	商品导出	products	GET /admin/products/export 导出商品列表为 Excel, 含商品名称/价格/库存/销量
REQ-103	订单列表查询	orders, order_items	GET /admin/orders?status=&user_keyword=&order_no=&date_range=&page= 多维筛选
REQ-104	订单详情查看	orders, order_items, customizations	GET /admin/orders/{id} 展示订单全部信息, 含刻字定制内容及预览图
REQ-105	订单状态管理	orders	PUT /admin/orders/{id}/status, 手动变更订单状态 (需权限 order:manage)
REQ-106	订单备注	orders	PUT /admin/orders/{id}/admin-remark, 管理员添加内部备注 (新增 admin_remark 字段)
REQ-107	批量发货	orders, logistics	POST /admin/orders/batch-ship, 上传 Excel 批量录入运单号, 批量创建 logistics 记录
REQ-108	单笔发货	logistics	POST /admin/orders/{id}/ship, 填写物流公司和运单号, orders.status: processing→shipped
REQ-109	发货物流录入	logistics	填写物流公司名称 (下拉+手动输入)、运单号, 保存 logistics 记录, shipped_at=NOW()
REQ-110	订单导出	orders, order_items	GET /admin/orders/export, 导出订单列表为 Excel, 支持按筛选条件导出
REQ-111	订单统计看板	orders	汇总今日/本周/本月订单数、待处理订单数、各状态分布 (聚合查询)
REQ-112	订单搜索	orders	支持按订单号、用户手机号、收件人姓名、商品名称等多字段搜索
REQ-113	物流信息管理	logistics	GET /admin/logistics?status=&date_range= 查询发货列表, 支持批量导出
REQ-114	物流轨迹同步	logistics.tracking_data	定时任务调用第三方物流API (快递鸟/快递100) 同步最新轨迹到 tracking_data
REQ-115	售后申请列表	after_sales	GET /admin/after-sales?status=&type=&date_range=&page= 管理所有售后申请
REQ-116	售后申请详情	after_sales, orders	GET /admin/after-sales/{id} 展示售后详情、原订单信息及凭证图片
REQ-117	售后审核	after_sales	PUT /admin/after-sales/{id}/review, 审核通过/拒绝, 填写 review_remark
REQ-118	微信退款操作	payments, after_sales	审核通过后自动调用微信退款 API, 等待退款回调更新 status
REQ-119	PayPal 退款操作	payments, after_sales	审核通过后自动调用 PayPal Refunds API, 同步更新退款状态
REQ-120	退款记录查询	payments	GET /admin/payments?status=refunded&method= 查询退款记录, 含退款金额和时间

需求编号	需求名称	对应表/模块	实现细节
REQ-121	支付记录列表	payments	GET /admin/payments?method=&status=&date_range=&page= 查询所有支付流水
REQ-122	支付记录详情	payments	GET /admin/payments/{id} 展示支付完整信息（微信交易号/PayPal订单ID/金额/时间）
REQ-123	汇率配置	exchange_rates, system_settings	POST /admin/settings/exchange-rate, 手动设置汇率，写 exchange_rates (source=manual) 并更新 Redis
REQ-124	汇率历史记录	exchange_rates	GET /admin/settings/exchange-rates/history 查询汇率变更历史，含自动获取和手动设置记录
REQ-125	微信支付对账	payments	GET /admin/payments/reconcile 按日期汇总微信支付流水与内部订单比对，标记差异
REQ-126	PayPal 对账	payments	GET /admin/payments/paypal-reconcile 汇总 PayPal 流水与内部记录比对
REQ-127	支付报表导出	payments	GET /admin/payments/export, 导出支付明细 Excel，按日期/支付方式分组汇总
REQ-128	销售数据概览	orders, order_items	仪表盘展示：今日/本周/本月销售额（CNY）、订单量、客单价，折线图趋势
REQ-129	商品销售排行	order_items	统计各商品/SKU销售量和销售额，按排名展示 TOP 10
REQ-130	用户增长统计	users	按日/周/月统计新注册用户数、校友认证通过数，折线图展示趋势
REQ-131	订单状态分布	orders	饼图展示各状态订单占比（待支付/已付款/处理中/已发货/已完成/已取消/退款中）
REQ-132	支付方式分布	payments	饼图展示微信支付 vs PayPal 支付的笔数和金额占比
REQ-133	地区分布统计	orders, addresses	按省份/国家统计订单量和金额，热力图展示
REQ-134	库存预警	product_skus	定时扫描 stock < 预警阈值（可配置）的 SKU，发送邮件/系统告警通知管理员
REQ-135	评价统计	reviews	统计各商品平均评分、好评率（4-5星占比），展示评价关键词云
REQ-136	售后数据统计	after_sales	统计售后申请数、退款总额、各类型比例、审核通过率
REQ-137	定制刻字统计	customizations, order_items	统计刻字定制订单量、常用字体、高频刻字内容（脱敏展示）
REQ-138	报表定时发送	report_schedules	按 report_schedules 配置，定时生成报表并发送邮件给指定收件人
REQ-139	Banner 管理	banners	CRUD Banner，设置图片/跳转链接/生效时间/排序，预览效果

需求编号	需求名称	对应表/模块	实现细节
REQ-140	公告管理	announcements	CRUD 公告，设置类型（普通/重要/紧急）、发布时间和过期时间
REQ-141	系统参数配置	system_settings	键值对配置管理界面，支持修改订单超时时间、刻字字数限制等系统参数
REQ-142	操作日志查询	admin_logs	GET /admin/logs?admin_id=&action=&date_range= 查询管理员操作日志
REQ-143	报表计划配置	report_schedules	CRUD 报表发送计划，设置频率（日/周/月）、收件人列表、报表类型
REQ-144	系统健康监控	-	展示服务器状态（CPU/内存/磁盘）、Redis连接状态、MySQL连接池状态、最近错误日志
REQ-145	数据备份配置	system_settings	配置 MySQL 自动备份策略（备份频率、保留天数、备份存储路径），展示最近备份记录

附录：数据库 ER 关系摘要

users (1) — (N) alumni_verifications
users (1) — (1) carts
users (1) — (N) addresses
users (1) — (N) orders
users (1) — (N) notifications
users (1) — (N) customizations
users (1) — (N) reviews
users (1) — (N) after_sales
users (1) — (N) payments

orders (1) — (N) order_items
orders (1) — (1) logistics
orders (1) — (N) after_sales
orders (1) — (N) payments

products (1) — (N) product_skus
products (1) — (N) product_images
products (N) — (1) product_categories（支持二级分类）

order_items (N) — (1) customizations
order_items (1) — (1) reviews

admins (N) — (N) roles [via admin_roles]
roles (N) — (N) permissions [via role_permissions]
admins (1) — (N) alumni_verifications (reviewer_id)
admins (1) — (N) after_sales (reviewer_id)

附录：技术栈说明

层次	技术选型	说明
小程序前端	微信小程序原生 / uni-app	支持跨端，海外用户 WebView 嵌入 PayPal 页面
后端框架	Python FastAPI / Node.js NestJS	RESTful API，JWT 鉴权
数据库	MySQL 8.0	主从复制，JSON 字段支持
缓存	Redis 7.x	库存缓存、超时队列、汇率缓存、Session 黑名单
对象存储	阿里云 OSS / 腾讯云 COS	图片/预览图存储，CDN 加速
消息队列	Redis 延迟队列 / RabbitMQ	订单超时、异步通知
图片处理	Python Pillow	刻字预览图渲染
支付	微信支付 APIv3 + PayPal Orders API v2	双渠道支付
物流	快递鸟 API / 快递100	物流轨迹同步
内容安全	微信内容安全 API	刻字内容敏感词过滤
部署	Docker + Kubernetes / 云服务器	容器化部署，水平扩展